

Predictive Maintenance Deployment Recommendation Using AWS

A proposal based on the documented Reliance Steel and Aluminum project with TensorIoT

Assignment 1

Aseem Syed
Student ID 100941386
October 9, 2026

Recommendation

I recommend a hybrid setup. Predictions and alerts would run at the plant, while AWS would handle training and historical data. I would delay the full launch until the model, and the local system have been tested in a small pilot.

GitHub commit link: [FraudGuard-AI/docs/assignment1/INFT41000_A1_Syed_Aseem.pdf at main · aseemsyed10/FraudGuard-AI](https://github.com/aseemsyed10/FraudGuard-AI/blob/main/docs/assignment1/INFT41000_A1_Syed_Aseem.pdf)

AI use statement

I used ChatGPT to help find sources, organize and draft the report, work through the proposed design and costs, and format the document. The company case is real, but the proposed plant size, scores, and costs are assumptions for this assignment. I did not have access to Reliance's internal records. I am responsible for checking the final work and its sources.

The completed readiness scorecard is in Appendix A, and the environment scorecard is in Appendix B. The risk example and framework discussion now use the supplied class slides.

1 Use case and readiness snapshot

1.1 Use case description

I chose the predictive maintenance project at Reliance Steel & Aluminum Co. because it shows how AWS can be used with industrial machinery. Reliance worked with TensorIoT to collect machine data and use Amazon SageMaker to help identify maintenance needs .

For this assignment, I use that case as the starting point for a proposed metals-processing facility. Technicians and supervisors would use alerts to decide which machines need inspection. I would use vibration, temperature, motor current, operating hours, and maintenance records as inputs. My proposed model is a gradient-boosted classifier that estimates whether failure-related maintenance will be needed within 24 hours. The algorithm and prediction window are my design choices, not details confirmed by Reliance.

Predictions would run on local gateways so alerts could continue without internet access. AWS would handle training and historical storage. I consider monitoring and maintenance the most important lifecycle stage because equipment wear and changing operating conditions can affect the model after deployment.

1.2 Why production AI is required

This needs to be a production system because technicians would rely on it during normal shifts. A missed warning could lead to a breakdown, while too many false alarms could waste time and make staff stop trusting it. It needs reliable alerts, monitoring, recovery, and clear ownership, while existing machine safety controls remain separate.

1.3 Readiness summary

Using the class template, I scored data 1, model 0, infrastructure 2, organizational readiness 1, risk 3, and cost 3. The total is 10/30, with an average of 1.67/5. These scores describe my proposed deployment, not Reliance's existing system. I assume 50 machines, 10 users, historical sensor data, and a small IT team. Appendix A includes the completed class form and explains how each score affects my design.

1.4 Weakest dimensions and their effect

I gave model readiness 0/5 because there is only a proposed model; there is no working model for this facility to assess. This follows the template's zero rating. The next step would be to build a baseline, test it on later data that was not used for training, and then run it in shadow mode. Shadow mode means recording predictions without changing maintenance decisions yet.

Data and organizational readiness are tied at 1/5. The assumed records have not been shown to pass validation checks, and no operational owner has been named. These gaps need attention before launch. The data needs cleaning and documentation, and someone must be responsible for the model and its alerts. AWS can reduce infrastructure work, but it cannot take over those responsibilities.

2 Deployment environment evaluation

2.1 Environment Fit Scorecard

The completed Environment Fit Scorecard is in Appendix B. Following the class template, I assessed continuous prediction and alerting as the most demanding workload. I gave data sensitivity 2 because machine data is commercially sensitive, although approved features could be uploaded. Cost pattern is 2 because serving demand is steady. Operational capacity is 4 because I assume the small IT team cannot support the ML platform around the clock without managed services. Latency is 1 because local alerts must continue during internet outages. Training is a separate, occasional workload.

The average is $(2 + 2 + 4 + 1) / 4 = 2.25$, and the spread is $4 - 1 = 3$. Since latency scores 1, public cloud alone is ruled out. The spread of 3 then points to hybrid deployment.

I applied the rules in the order given in class. Rule A checks the hard constraint first. Rule B selects hybrid when the spread is 3 or more. Only otherwise would Rule C use the average: 2.4 or less means on-premise, 3.6 or more means cloud, and the middle range uses operational capacity to break the tie. In this case, the average of 2.25 does not override Rule B.

2.2 Comparison of the three environments

An on-premise setup would keep all data and predictions at the plant. It would also work without an internet connection. The drawback is that the plant would need to buy and maintain its own training server, storage, backups, and serving system. Since training is only occasional, some of that equipment would sit idle.

A cloud-only setup would make it easier to add training and storage capacity. However, sending every prediction to the cloud would make alerts depend on the plant's internet connection. Even a working AWS service cannot return a useful alert if the plant cannot reach it. This option does not meet the outage requirement.

A hybrid setup keeps the urgent work local and puts work that can wait in AWS. Its main drawback is the extra coordination between the two environments. The team needs to manage model versions, stored data waiting for upload, and monitoring in both places. Appendix B compares the three options across the required dimensions, following the Week 3 framework.

2.3 Recommendation

I recommend hybrid deployment with local inference and alerts, and AWS-based model training, historical feature storage, and release management.

Latency and operational capacity are the main reasons for my choice. Feature processing, predictions, the dashboard, and alert logs would stay at the plant. Approved features and maintenance labels would be uploaded for training. This avoids a permanent cloud prediction endpoint and lets local alerts work independently. AWS documents that Greengrass can run cloud-trained models on local data.

I am adapting the real case to the assignment's requirements. The published Reliance design describes cloud inference with results sent back to machinery. I recommend local inference because the assignment specifically says a bad network connection must not stop alerts. This recommendation also follows the class scorecard rule.

2.4 Three year cost reasoning

I compared hybrid AWS and fully on-premises deployment over three years in Appendix C2. The estimated totals are C\$38,957.22 for hybrid and C\$78,243.10 for on-premises. These are planning estimates for my proposal, not Reliance's actual costs. AWS charges come from the October 9, 2026 calculator estimate. I converted USD amounts at US\$1 = C\$1.4240, the Bank of Canada rate for October 8, 2026.

For the estimate, I assumed 50 machines making one prediction per second. That is about 129.6 million local predictions in a 30-day month. I included two gateways: one active and one standby. Instead of uploading every raw reading, the system would send summaries and selected diagnostic records, estimated at 25 GB per month. I budgeted 1,000 GB of average retained cloud data, including earlier records and model files.

I allowed 20 CPU training hours per month on one SageMaker ml.m4.4xlarge instance. The calculator uses US\$0.96/hour, giving US\$19.20/month for training. Temporary training storage adds about US\$0.19/month. I would start with CPU training because the proposed classifier does not yet justify a GPU; instance suitability and availability would be checked before deployment.

The AWS estimate is US\$65.65/month, or C\$3,365.48 over 36 months. It includes S3 storage and transfers, two active Greengrass cores and supporting IoT connections, Lambda batch validation, and CloudWatch monitoring. Lambda free-tier discounts are excluded; the calculator includes the free CloudWatch dashboard allowance. I assume constant usage, prices, and exchange rate, with no promotional credits or paid support plan.

Staff time is the least certain part of my estimate. I assumed 10 hours per month for hybrid and 20 hours for on-premises, at C\$85.44 per hour, including employment costs. If hybrid also needs 20 hours per month, its three-year total rises from C\$38,957 to about C\$69,716. This reduces the estimated saving compared with on-premises from C\$39,286 to about C\$8,527, so most of the saving depends on staff effort. I would measure support time and data uploads during the pilot before relying on these savings.

2.5 Tipping point

I would switch to fully on-premise if a binding company requirement stopped any machine-derived data, including summary features, from leaving the facility. In that situation, AWS could not train on the plant's data, so the main benefit of the proposed cloud setup would disappear.

3 Cloud platform and service selection

3.1 Why AWS

I chose AWS because it fits both the real case and the proposed workload. Reliance's project provides an example of AWS IoT and SageMaker being used together. Python and Docker also fit the proposed development approach, and Greengrass supports container-based edge components. Managed training and model registration would reduce setup work for a small team. Paying for occasional training suits this workload, although it does not automatically make AWS the cheapest option.

I did not choose Azure because this scenario has no existing Microsoft cloud setup that would give it a clear advantage. I did not choose Google Cloud because the selected AWS services already fit the proposed edge and training workflow. Both are reasonable alternatives; my choice is based on this project rather than a claim that AWS is always better.

3.2 Service selection and responsibilities

Appendix C1 gives the full service map. SageMaker AI would train the model, Model Registry would track versions and approvals, and S3 would store approved data and model files. Greengrass V2 would manage local components. Lambda would check uploaded batch information, CloudWatch would collect logs and health information, and IAM would control cloud permissions .

Using NIST's service-model definitions, I classify the managed application platforms as PaaS and S3 as infrastructure storage. Greengrass provides a PaaS-style management layer, but the plant still owns and maintains the gateways. These labels help explain which responsibilities stay with the plant.

3.3 Containers or serverless

For local inference, the traffic is continuous and the process needs to stay running during production. It should also be easy to move to a replacement gateway. I would therefore use Docker for feature processing and predictions. The dashboard would use local access controls so staff would not need a cloud login during an outage.

For upload checks, traffic arrives in batches and each task should take only seconds. I would use Lambda because it suits this short, event-based work. The checking logic would stay in a separate Python module to make it easier to move later. I would process uploaded batches rather than invoke a function for every individual sensor reading.

Training happens occasionally but may run for minutes or hours, so I would use container-based SageMaker jobs. The container keeps the software dependencies consistent. AWS manages the underlying training infrastructure, which reduces the amount of server maintenance the team must do.

3.4 Build versus buy decision

My choice is to build the model, feature-processing code, and local alert application, while using managed training and model-management services. SageMaker would reduce setup time and still let the team use its own model code. A self-managed server offers more control and could suit heavy, steady use, but it also adds patching, backups, scheduling, and recovery work. For the assumed small team and occasional training, I think managed infrastructure is more practical. The team would still need to understand the machines and validate the model.

3.5 Adoption and ongoing operation

Plan

I would begin by identifying the machines whose breakdowns cause the most disruption. I would record current downtime and maintenance effort, agree on what counts as a failure, and check whether a 24-hour warning window is useful. The plant would also need a sponsor, a model owner, an approved pilot budget, and rules for cloud uploads. I use the Plan, Ready, Adopt, and Govern & Operate stages from Week 4.

Ready

Before starting, I would separate development and production AWS accounts, require multifactor authentication, and limit each role to the permissions it needs. I would set up private S3 access, encrypted connections, log retention, and budget alerts. Those alerts notify the team but do not guarantee a spending cap. At the plant, gateways would be separated from machine control networks, with individual credentials, a patch schedule, local operator access, and tested backups.

Adopt

I would start with five machines in shadow mode and compare predictions with inspections and actual failures. Training and evaluation data would be separated by time so later information could not leak into earlier predictions. I would also check results for different machine types and operating conditions. The pilot would test internet loss, gateway restart, uploads after reconnection, duplicate records, rollback, and standby recovery. Expansion would depend on the criteria in Section 4.

Govern and Operate

Maintenance staff would respond to alerts, IT would maintain the gateways, and the model owner would review results and approve updates. I would check model performance, data quality, and costs monthly, with earlier reviews when problems appear. Each gateway would keep a working previous model. Training data versions, test results, and release decisions would also be recorded. This follows Huyen's emphasis on maintaining the whole ML system after deployment [2].

3.6 Lock in and mitigation

The parts most closely tied to AWS are Greengrass recipes, SageMaker management interfaces, IAM policies, and Lambda event connections. Moving to another provider would require changes to these parts. To reduce that work, I would keep the model code separate from AWS-specific code, use Docker, and store data in documented CSV or Parquet formats. I would also test that an exported model can be restored locally.

Offline behavior and trade offs

During an internet outage, the local system would keep using its last approved model. Cloud updates and uploads would wait. I would provide a disk buffer for at least 72 hours of upload data at the assumed rate and use sequence IDs to avoid duplicate uploads. If the buffer fills, the system would keep alert records first and tell operators what data was dropped. The trade-off is extra gateway maintenance and sometimes using an older model.

4 Risks and decision

4.1 Three priority risks

Availability failure, likelihood medium, impact high. Losing internet access would stop cloud synchronization, while a failed gateway could stop local predictions. I would keep the model and alerts local, buffer uploads on disk, retry safely, and test recovery on a standby gateway. The live alert process would not need a cloud connection.

The example in Week 4, slide 12 is the December 2021 AWS US-EAST-1 outage. The slide describes a network problem that disrupted major services for hours [11]. For my design, the lesson is that a cloud outage should not stop local maintenance alerts. I would keep the approved model, dashboard, and alert process at the plant and let cloud uploads and updates wait until service returns.

Model error or drift — likelihood high, impact high. A faulty sensor, a different machine load, or incorrect labels could lead to missed failures or too many alerts. I would test on later holdout data, check each machine type, and monitor data quality and alert rates. New models would need approval, and the previous version would remain available. Information recorded after a failure would not be allowed into the features used to predict that failure.

Unauthorized access or updates — likelihood medium, impact high. Stolen gateway credentials or overly broad permissions could expose data or allow an unsafe model update. I would use separate device identities, limited permissions, encrypted traffic, private storage, artifact integrity checks, and regular patching. AWS manages its service infrastructure, but the plant remains responsible for its access settings, devices, application, and data handling. Week 3, slide 10 uses the Capital One breach to illustrate this shared-responsibility issue.

4.2 Decision and conditions

My decision is to delay the full production launch. First, the team should fix the data and ownership gaps, build a baseline model, and complete a controlled pilot. I would record Ship only after the conditions below are met. This decision applies to my proposed deployment, not Reliance's existing system.

Data quality: at least 95% of pilot records pass agreed timestamp, machine-ID, and sensor-validity checks.

Model usefulness: proposed targets are at least 90% event-level recall within the defined warning window and no more than one false alert per machine per week. Report independent event counts and uncertainty; insufficient failure evidence means continuing the pilot.

Availability and performance: pass a 24-hour internet-loss test and achieve a 99th-percentile alert delay below one second after each feature window becomes available.

Recovery and ownership: recover via the standby within five minutes, demonstrate model rollback, assign operational owners, and complete access-control checks.

Cost and safety: measure pilot staff time and cloud usage against the approved budget; retain independent existing machine safety controls.

5 Communication and professional practice

5.1 Deployment diagram

Figure 1 shows how I would split the system. The machines, predictions, and technician alerts stay on the plant network. Data uploads and model downloads happen when the connection is available. IAM manages AWS access, while local access stays under plant control. Some monitoring and permission links are left out to keep the main data flows readable.

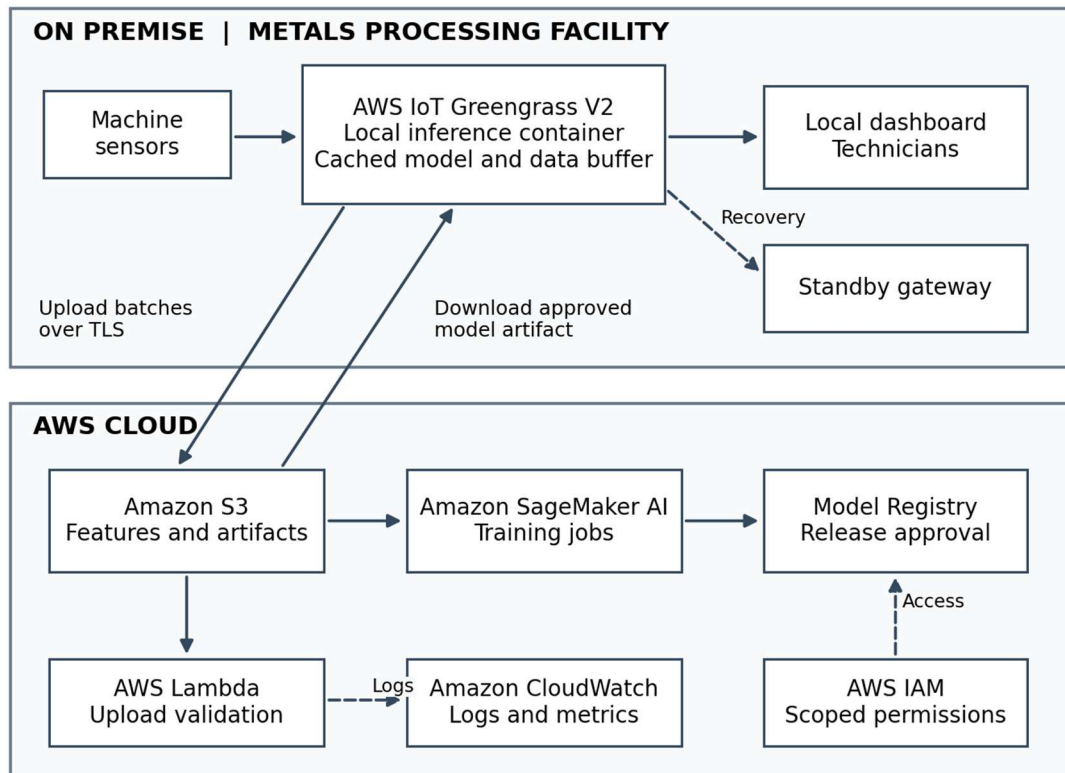


Figure 1. My proposed hybrid deployment. The standby gateway provides a recovery option; I am not assuming instant failover. Gateway health information is sent to CloudWatch when connected, while local checks continue offline.

5.2 Version control and submission

I committed the report PDF to docs/assignment1/INFT41000_A1_Syed_Aseem.pdf in my course GitHub repository. The commit records the uploaded report, and further updates will be saved as separate commits to preserve its revision history. Credentials, secret keys, and private machine records are not included.

5.3 References

Sources accessed October 9, 2026. The company case is historical; the workload, scores, and non-AWS cost allowances describe my proposal.

- [1] D. De Alday and L. Tran, “Bringing Intelligence to Industrial Manufacturing Through AWS IoT and Machine Learning,” AWS Partner Network Blog, Nov. 4, 2019.
<https://aws.amazon.com/blogs/apn/bringing-intelligence-to-industrial-manufacturing-through-aws-iot-and-machine-learning/>
- [2] C. Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. O’Reilly Media, 2022. Author resources: <https://github.com/chiphuyen/dmls-book>
- [3] P. Mell and T. Grance, The NIST Definition of Cloud Computing, NIST SP 800-145, 2011.
<https://doi.org/10.6028/NIST.SP.800-145>
- [4] AWS, AWS IoT Greengrass V2 Developer Guide, “Perform machine learning inference” and “Run a Docker container,” n.d.
<https://docs.aws.amazon.com/greengrass/v2/developerguide/perform-machine-learning-inference.html> ; <https://docs.aws.amazon.com/greengrass/v2/developerguide/run-docker-container.html>
- [5] AWS, “Amazon SageMaker AI pricing,” n.d. <https://aws.amazon.com/sagemaker/ai/pricing/>
- [6] AWS, “Amazon S3 pricing” and “AWS pricing,” n.d. <https://aws.amazon.com/s3/pricing/> ; <https://aws.amazon.com/pricing/>
- [7] AWS, “AWS IoT Greengrass pricing,” n.d. <https://aws.amazon.com/greengrass/pricing/>
- [8] AWS, “AWS Lambda pricing,” n.d. <https://aws.amazon.com/lambda/pricing/>
- [9] AWS, “Model Registration Deployment with Model Registry,” n.d.
<https://docs.aws.amazon.com/sagemaker/latest/dg/model-registry.html>
- [10] AWS, service documentation, n.d.: “What is IAM?”
<https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html> ; “What is Amazon CloudWatch?”
<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html> ; “What is AWS Lambda?” <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html> ; “What is Amazon S3?” <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>
- [11] O. Altrad, Week 4 — Cloud Platforms and AI Infrastructure, INFT 41000 lecture slides, Durham College, n.d., slides 10–12 and 17. Course-provided PDF.
- [12] O. Altrad, Week 3 — AI Deployment Environments, INFT 41000 lecture slides, Durham College, n.d., slides 10 and 16. Course-provided PDF.
- [13] AWS, “AWS Pricing Calculator: predictive-maintenance estimate,” Oct. 9, 2026.
<https://calculator.aws/#/estimate?id=bee84d96f48d720f6ae87f5e06e2d273ed7af355>
- [14] Bank of Canada, “Daily exchange rates,” Oct. 8, 2026. USD/CAD: 1.4240.
<https://www.bankofcanada.ca/rates/exchange/daily-exchange-rates/>

Appendix A Readiness scorecard

The completed Week 2 class scorecard follows this summary as part of Appendix A. The table explains what each score means for my design. I am assessing the proposed facility, not Reliance's internal system, so I have not treated planned work as completed testing.

Dimension and score	Evidence and gap classification	Effect on deployment
Data 1/5	Historical telemetry is assumed, but no validation results or documented lineage are available. Fix before launch.	Validate and version data; resolve maintenance labels before training.
Model 0/5	Only a proposed classifier exists; no implemented target-facility model can be assessed. Fix before launch.	Build a baseline, then validate before shadow deployment.
Infrastructure 2/5	Sensor collection and a plant network are assumed; the serving stack and recovery are not deployed or tested. Fix before launch.	Implement local serving, monitoring, buffering, and recovery.
Organizational 1/5	No accountable operational model owner is named in the proposal. Fix before launch.	Assign ownership and response duties; document escalation.
Risk 3/5	Risks are identified in this report, but the proposed mitigations are not implemented or verified. Fix before launch.	Test access controls, offline behavior, and rollback.
Cost 3/5	A three-year estimate exists, but avoided downtime value has not been quantified. Fix before launch.	Compare costs with expected value and obtain budget approval.

Total 10/30; average 1.67/5. Lowest dimension: model 0. Data and organizational readiness tie at 1. Any score of 0 or 1: Yes. Decision: Delay production for remediation and baseline development, followed by a controlled pilot. Section 4 defines release criteria.

I have marked cost as a fix-before-launch gap because I have estimated spending but have not calculated the value of avoided downtime. The sponsor would need to compare those figures and approve the pilot budget. After launch, the team would continue tracking actual costs and support time.

Production Readiness Scorecard

Production Readiness Scorecard

Use case	Predictive maintenance proposal based on Reliance			Prepared by	Aseem Syed	Date	October 9, 2026
Scoring scale	0 Nothing exists, or cannot be assessed	1 Red flag See the rubric anchors below	2 Between a 1 and a 3	3 Developing See the rubric anchors below	4 Between a 3 and a 5	5 Ready See the rubric anchors below	
Dimension and guiding question	Rubric anchors		Score (tick one)	Evidence: one specific sentence		Type of gap	
1. Data <i>Is our data trustworthy enough to run a business decision on?</i>	1 No validation, unknown lineage 3 Validated, but lineage undocumented 5 Validated, versioned, governed		1 / 5 selected	Telemetry is assumed, but validation and lineage are not demonstrated.		Fix before launch	
2. Model <i>Does the model perform reliably outside training conditions?</i>	1 Never tested outside training data 3 Tested once, no bias/drift plan 5 Tested, explainable, monitored for drift		0 / 5 selected	No implemented target-facility model exists; only a classifier proposal is available.		Fix before launch	
3. Infrastructure <i>Can our systems handle real load, failure, and change?</i>	1 Runs on someone's laptop 3 Deployed, but no monitoring/rollback 5 Containerized, monitored, autoscaled, recoverable		2 / 5 selected	Collection/network are assumed; serving, monitoring, and recovery are not deployed.		Fix before launch	
4. Organizational <i>Do we have the people and process to run this responsibly?</i>	1 No named owner 3 Owner named, no documented process 5 Owner, on-call, governance all documented		1 / 5 selected	No operational model owner is named for the proposed facility.		Fix before launch	
5. Risk <i>What's the worst case if this goes wrong, and can we live with it?</i>	1 No risk review done 3 Risks identified, not mitigated 5 Risks identified, mitigated, and monitored		3 / 5 selected	Risks are identified, but mitigations remain unimplemented and untested.		Fix before launch	
6. Cost <i>Does the value this creates outweigh what it costs to keep alive?</i>	1 Cost never estimated 3 Cost estimated, not compared to value 5 Cost tracked and clearly justified by value		3 / 5 selected	Costs are estimated, but avoided downtime and maintenance value are unquantified.		Fix before launch	
Total: 10 / 30		Average: 1.67 / 5		Lowest: Model (0); Data and Organization (1)		Any score of 0 or 1? YES	
Go / no-go decision		DELAY for remediation					
Conditions and next actions (owner and date)		Proposed owners/dates: sponsor appoints model owner by pilot start; data lead validates records before training; ML lead validates baseline before shadow mode; IT lead tests offline/recovery/security before release; sponsor approves cost/value before launch. No appointments or dates are confirmed.					
Reviewed by (name, role, date)		Aseem Syed — student — October 9, 2026. Academic assessment; no company reviewer or approval claimed.					

Appendix B Environment Fit Scorecard

Question	Evidence or assumption	Score
1 Data sensitivity	Commercial telemetry; approved features may leave the plant.	2
2 Cost pattern	Continuous prediction and alerting has steady demand; training is scored separately in the workload split.	2
3 Operational capacity	General IT can support devices but lacks specialist ML platform coverage.	4
4 Latency	Internet outages must not stop local alerts.	1

The completed Week 3 class scorecard follows this summary in Appendix B. Serving scores: 2, 2, 4, 1. Average 2.25; spread 3. Data or latency scored 1: Yes. Rule A rules out public cloud alone. Rule B selects hybrid at spread ≥ 3 . Rule C is not reached; otherwise ≤ 2.4 selects on-premise, ≥ 3.6 cloud, with operational capacity breaking intermediate ties.

Environment comparison

Dimension	On premise	Cloud	Hybrid
Data control	All telemetry and models stay at plant.	Uploaded data resides in AWS.	Raw streams local; approved features uploaded.
Cost model	Buy training, serving, and storage capacity.	Pay for training, storage, and online serving.	Buy gateways; pay for occasional cloud work.
Scalability	Procure more plant hardware.	Increase cloud resources.	Scale training in AWS; add gateways locally.
Operations	Maintain the complete stack.	Manage models and cloud configuration.	Maintain gateways and cloud integration.
Best fit	Offline capable; larger local support burden.	Fails offline-alert requirement.	Meets offline need with managed training.

Recommendation agrees with the official rule. Local serving meets the hard constraint; bursty cloud training provides the explicit workload split.

INFT 41000 | WEEK 3 | AI DEPLOYMENT ENVIRONMENTS

Environment Fit Scorecard Four questions, scored 1 to 5: on-premise, cloud, or hybrid?

Use case	Predictive maintenance proposal based on Reliance		Prepared by	Aseem Syed	Date	October 9, 2026
Scoring scale	1 Strong on-premise pull Hard limit or clear need	2 Leans on-premise	3 Mixed Either could work	4 Leans cloud	5 Strong cloud pull Cloud clearly fits	
Question (ask in this order)	Anchors: what each score looks like		Score (tick one)	Evidence: one specific sentence		
1. Data sensitivity <i>Can this workload's data leave our own network?</i>	1 Regulated or highly sensitive; cannot leave our network 3 Mixed: some regulated data, some not 5 Low sensitivity or de-identified; provider certifications suffice		2 / 5 selected	Commercial telemetry may be uploaded only as approved features.		
2. Cost pattern <i>Is demand steady enough to justify owning hardware, or bursty enough to rent it?</i>	1 Steady, predictable, high volume; owning hardware pays off 3 Steady baseline with occasional peaks 5 Unpredictable or bursty; pay-as-you-go beats idle hardware		2 / 5 selected	The scored serving workload has steady, continuous demand; bursty training is split to AWS.		
3. Operational capacity <i>Do we have the in-house team to run this 24/7, and do we want to?</i>	1 Strong in-house team can run it around the clock 3 Some skills, gaps in on-call or specialist roles 5 No in-house team; the provider must manage the hardware		4 / 5 selected	The assumed small IT team lacks specialist ML platform and on-call coverage.		
4. Latency <i>How much delay can this system tolerate between data and compute?</i>	1 Near-zero tolerance; must run beside the data or on site 3 Moderate tolerance; some network delay is acceptable 5 Delay-tolerant; batch or asynchronous work is fine		1 / 5 selected	Local prediction and alerts must continue during internet outages.		
Average: 2.25 / 5		Spread: 4 - 1 = 3		Data or latency scored 1? YES		Ruled out: public cloud alone
Recommendation		HYBRID — local serving; cloud training				
One-sentence justification		Latency 1 rules out cloud-only serving and spread 3 selects hybrid: local alerts continue offline while AWS provides intermittent training.				
The question that drove the decision		Latency (hard limit), then operational capacity (wide spread). Rule C is not reached.				

Appendix C1 AWS service map

This is my proposed service map rather than a copy of every service in the original Reliance project. The local gateways and disk buffers would remain the plant's responsibility. Greengrass uses supporting AWS IoT Core connections for device management; these connection costs are included in the estimate [13].

Building block and service	Model	Execution	Reason
Training compute Amazon SageMaker AI	PaaS	Container jobs	Intermittent CPU training without a dedicated server.
Serving AWS IoT Greengrass V2	PaaS style edge layer	Local Docker	Cached approved model and local alerts during outages.
Data storage Amazon S3	IaaS storage	Managed storage	Approved features, labels, and versioned artifacts.
Managed ML SageMaker Model Registry	PaaS	Managed control	Track versions, evaluation metadata, and approval.
Upload validation AWS Lambda	PaaS	Serverless	Short schema and metadata checks on batch arrival.
Monitoring Amazon CloudWatch	PaaS	Managed service	Cloud logs and delayed gateway health reporting.
Access AWS IAM	PaaS support	Managed control	Scope cloud permissions by workload and role.

Serving is continuous and portability matters, so containers are selected. Cloud validation is short and bursty, so Lambda is selected. Training is long-running but occasional, so managed container jobs are selected. NIST classifications are applied by responsibility boundary [3].

The dashboard would use local access controls so it remains available without a cloud login. Model Registry approval would not install a model by itself. An authorized release process would update the Greengrass deployment and download the approved file from S3. If an update fails, the last working model would stay available.

Appendix C2 Three year cost estimate

CAD; 36 months; US East (N. Virginia). AWS Calculator estimate dated October 9, 2026 , converted at US\$1 = C\$1.4240 [14]. Hardware, staffing, electricity, and local support are planning allowances, not quotations. H = high uncertainty; M = medium.

Cost line	Assumption and source (Oct. 9, 2026)	Hybrid CAD	On-premises CAD	H/M/L
Compute	Two gateways at C\$2,136 each; on-premises adds a C\$5,696 server. Includes power and hybrid training/storage at US\$19.39/month [13].	\$5,827.35	\$11,315.10	M
Storage and transfer	S3: 1,000 GB average, 10,000 writes and 10,000 reads/month; 25 GB in and 10 GB out/month [13]. Local backup allowance: C\$2,848.	\$1,227.77	\$2,848.00	M
Managed services / licences	Greengrass and IoT connections, Lambda, and CloudWatch [13]. Local support allowance: C\$854.40/year.	\$1,143.70	\$2,563.20	M
Staff time	Hybrid: 10 hours/month; on-premises: 20 hours/month. Assumed C\$85.44/hour including employment costs, for 36 months.	\$30,758.40	\$61,516.80	H
Three-year total	Constant usage, prices, and exchange rate; totals based on displayed calculator monthly costs.	\$38,957.22	\$78,243.10	H

AWS usage: training storage is 50 GB for 20 hours/month (1.37 GB-months). Lambda uses 10,000 requests/month × 5 seconds × 1 GB. CloudWatch assumes 50 metrics, 10,000 other API requests, 10 GB logs with one-month retention, one dashboard, and 10 standard alarms. Its free dashboard allowance is included; Lambda free-tier discounts are excluded .

Scope: 50 machines, 10 users, and two gateways active for billing. Power assumes 100 W hybrid or 240 W on-premises × 26,280 hours at C\$0.2136/kWh (about C\$561 and C\$1,347). If hybrid needs 20 support hours/month, its total rises to about C\$69,716 and the saving falls to C\$8,527. Pilot measurements must validate support time and uploads.

Excluded common costs: initial software development and integration, existing sensors, internet service, and taxes. No dedicated leased line, NAT gateway, premium support plan, or GPU is included. Reprice if any becomes necessary; obtain hardware quotes before financial approval.